

Proposal Approval Package — SafeExec

CISC 699 Applied Project, Summer 2026 — Hard Stop 1 (Week 2)

Zixuan Liang · zliang1@my.harrisburgu.edu

Submitted 2026-05-21 · Canvas due 2026-05-24

Contents

Part 1 — Project Proposal (`proposal.md`)

1. Executive summary
2. Background and significance
3. Problem statement
4. Research questions and hypotheses
5. Objectives
6. Intended artifact
7. Success criteria (measurable)
8. Scope (in / out / scope-fence)
9. Technical feasibility — MACP framing
10. Project plan (phases, milestones, Gantt, critical path, contingencies)
11. Work breakdown structure (summary; full backlog in Part 3)
12. Risk register
13. Assumptions and constraints
14. Ethics, privacy, security, broader impact
15. Stakeholders
16. Reproducibility plan
17. AI-use disclosure for this proposal
18. Approval

Part 2 — Project plan (Appendix C, `project-plan.md`) — phase overview, Gantt source, completion gates, critical path, dependency view, resource plan.

Part 3 — Work breakdown structure (Appendix B, `wbs.md`) — per-task estimate, week, deliverable, success-criterion traceability, blocking dependencies, totals.

Part 4 — One-page approval brief (Appendix A, `approval-brief.md`) — what's approved, what's conditional, what evidence is expected by W4.

Part 5 — Recorded walkthrough script (Appendix D, `walkthrough-script.md`) — ~15-minute script for the 50-pt rubric item.

Project Proposal — SafeExec

A Hardened, Threat-Modeled Python Execution Sandbox for LLM-Agent Tool-Use

AI-use disclosure. Drafted with Claude Sonnet 4.6 (Cowork desktop app). AI-drafted, student-revised. Key human-authored decisions in this document: every scope boundary in §8; every numeric success-criterion threshold in §7; every risk likelihood/impact rating and mitigation owner assignment in §12; the W7 explicit scope-renegotiation checkpoint as the project’s load-bearing relief valve; the contingency milestone dates added in §10.5; the framing of the adversarial test suite as the project’s intellectual contribution. Full audit trail: docs/ai-use-log.md.

Document control

Field	Value
Document	CISC 699 Proposal Approval Package (Hard Stop 1)
Working title	SafeExec
Student	Zixuan Liang (zliang1@my.harrisburgu.edu)
Program	M.S. Computer Information Sciences, Harrisburg University
Course	CISC 699-50-A-2026/Summer — Applied Project in Computer Information Sciences
Supervisor	Dr. Majid Shaalan
Term	2026-05-09 — 2026-08-14 (14 weeks)
Proposal version	1.0
Proposal date	2026-05-21
Submission target	2026-05-24 (Canvas, before 11:59pm ET)
Approval target	2026-05-26 (post supervisor review)
Supersedes	docs/01-launch-packet/project-charter.md (W1 charter; retained as historical record)

1. Executive summary

LLM-powered software agents — Claude Code, Code Interpreter, Cursor, Devin, and a growing open-source ecosystem — increasingly execute model-generated Python code as part of their tool-use loop. In production, that execution layer is the part most often built quickly and revisited never; the open-source landscape today consists of either heavyweight commercial platforms or lightweight wrappers with weak isolation

guarantees and no published threat models tied to reproducible benchmarks. SafeExec closes a measurable slice of that gap. It is a small HTTP execution service that accepts Python source and runs it inside an isolated environment, with two pluggable isolation back-ends — a hardened Docker configuration and a gVisor-backed configuration — exposed through a single `POST /execute` API. The intellectual contribution is a curated **adversarial test suite** of ≥ 40 programs across ≥ 6 categories with documented expected-contained-outcome labels, plus an empirical comparison of isolation strength and performance overhead across the two back-ends, reported with 95% confidence intervals. The artifact ships with a documented threat model, ≥ 100 functional test programs, a performance benchmark harness, reproducibility materials (pinned Dockerfiles, Makefile targets, deterministic test runner), a minimal reference agent for end-to-end demo, and a 15–25 page graduate-level technical report. Project supervisor approval is requested at the scope described in §8.

2. Background and significance

2.1 Why this problem matters now

Agentic systems built on large language models rely on a tool-use loop in which the model proposes code, an executor runs it, and the result is fed back into the model's next step. The executor is the layer at which model-generated text becomes program execution on real infrastructure. Today's open-source agents commonly execute model output with isolation that ranges from “an unsandboxed subprocess inside the agent's own process tree” to “a Docker container with default settings and a permissive network policy.” Neither is a defensible posture when the code being executed was produced by a language model under conditions of possible prompt injection.

This is not a hypothetical concern. The 2025 NIST AI 100-2 update [1] and the 2025 OWASP Top 10 for LLM Applications [2] both identify tool-execution isolation as an under-specified mitigation surface, with OWASP LLM05 (“Improper Output Handling”) and LLM06 (“Excessive Agency”) naming sandboxed execution explicitly as a standard mitigation. Real-world container-escape vulnerabilities continue to be discovered in widely deployed runtimes (CVE-2019-5736 [7], CVE-2024-21626), confirming that “default Docker” is not a defensible isolation posture against adversarial code.

2.2 What already exists, and where the gap is

Closed-source services (Anthropic's analysis tool, OpenAI's Code Interpreter) address these risks behind opaque infrastructure and publish almost nothing about their isolation models. The open-source landscape consists of one strong commercial-backed option — E2B, which uses Firecracker microVMs [5] — and a handful of lighter-weight projects that ship convenience but not measured isolation strength. None of the open-source projects in this space publishes a documented threat model tied to a reproducible adversarial benchmark. That is the specific gap this project addresses: not a new isolation primitive, but a **measurement framework** that lets the isolation strength of a sandbox be evaluated empirically rather than asserted.

2.3 Why this is appropriate for CISC 699

The problem produces all the components a CISC 699 applied project must produce: (a) a working computational artifact with non-trivial systems content, (b) an empirical evaluation with explicit statistical structure, (c) a defensible ethics/security/broader-impact story grounded in named standards (NIST AI 100-2, OWASP LLM Top 10), and (d) reproducibility artifacts that can be exercised by a reader on commodity hardware. It is bounded enough that a single graduate student can credibly close the named gap in fourteen weeks (Python-only, single-tenant, no network, no GPU), and it is *not* a tutorial replication — the adversarial suite and its methodology are original work.

3. Problem statement

LLM-powered software agents execute model-generated Python code with isolation primitives that were not designed for adversarial workloads. The open-source landscape lacks a code-execution sandbox that pairs (a) production-realistic isolation with (b) a documented threat model and (c) a reproducible adversarial benchmark that lets isolation strength be measured. SafeExec builds that sandbox for the narrowly scoped case of Python-only, single-tenant, no-network, no-GPU workloads on a Linux host, and contributes the threat model and adversarial benchmark as the project's central intellectual artifact.

What the project explicitly does *not* claim to solve. Defense against nation-state-grade adversaries, side-channel timing attacks, Spectre/Meltdown-class or hardware-level attacks, multi-tenant security between mutually distrusting users sharing one instance, network-level isolation for agents that legitimately need outbound network access, GPU-accelerated workload sandboxing, and replacement for trained-model alignment or input-validation defenses. The sandbox is the last line, not the only line.

4. Research questions and hypotheses

The project is framed by three measurable research questions. Each is paired with a hypothesis stated in advance so that the W9 results section can report whether the hypothesis was supported, refuted, or returned a null result.

#	Research question	Hypothesis	How it will be tested
RQ1	How does isolation strength differ between hardened-Docker and gVisor back-ends when measured against a curated adversarial suite?	gVisor will contain ≥ 5 percentage points more of adversarial programs than hardened Docker, with the difference concentrated in syscall-abuse and host-enumeration categories.	Adversarial suite (≥ 40 programs, ≥ 6 categories) run under each back-end; contained-outcome labels compared per program.

RQ2	What is the performance overhead of stronger isolation, measured as steady-state per-request latency on the functional suite?	gVisor will impose a 1.5–4× latency penalty over hardened Docker for short-running Python programs, consistent with prior gVisor case studies.	Performance benchmark sweep on the functional suite under each back-end; ≥30 samples per condition; 95% confidence intervals on the median.
RQ3	Is the trade-off between isolation strength (RQ1) and overhead (RQ2) navigable by configuration, or is it a hard choice?	The two back-ends will occupy distinct points on the isolation-vs-overhead curve, not a Pareto-equivalent region — i.e., there is a real trade-off, not a configuration mistake.	Joint plot of contained-outcome rate and median latency for both back-ends; visual inspection plus reported per-category breakdown.

The hypotheses are deliberately pre-registered. Null results (e.g., gVisor not significantly stronger on the adversarial suite, or overhead inside noise) will be reported as findings rather than failures, per the W9 plan in §10.

5. Objectives

5.1 Primary objectives

1. Deliver a working sandbox execution service with the two named isolation back-ends, satisfying the functional suite at ≥99%.
2. Author and publish an adversarial test suite of ≥40 programs across ≥6 categories with documented expected contained-outcome labels.
3. Produce an empirical isolation-vs-overhead comparison across the two back-ends with statistical confidence intervals (95% CI, ≥30 samples per condition).
4. Produce a 15–25-page graduate-level technical report meeting Level 1 (Advanced/Exceptional) on all rubric dimensions.

5.2 Secondary objectives

5. Document a threat model that names the adversary capabilities the sandbox defends against and the ones it does not.
6. Ship a minimal Claude-API-driven reference agent demonstrating end-to-end tool-use integration.
7. Maintain reproducibility such that a reader on a stock Ubuntu 22.04/24.04 host can execute `make setup` && `make test` && `make bench` and obtain reported numbers within ±10%.

5.3 Stretch objectives (only if ahead at W10)

8. Add a Firecracker microVM back-end as a third comparison point (per [4]).
9. Release the adversarial benchmark as a standalone open-source artifact with its own README and citation.

6. Intended artifact

The project delivers a single integrated artifact comprising ten components, each tied to one or more objectives in §5.

#	Component	Repo path	Maps to objective
A1	Sandbox execution service	src/	01, 06
A2	Hardened Docker back-end (W5–W6)	src/backends/ docker_hardened/	01, 05
A3	gVisor back-end (W7–W8)	src/backends/ gvisor/	01, 05
A4	Threat model document	docs/design/ threat-model.md	05
A5	Functional test suite (≥100 programs)	tests/ functional/	01
A6	Adversarial test suite (≥40 programs, ≥6 categories)	tests/ adversarial/	02 — central contribution
A7	Performance benchmark harness	benchmarks/	03
A8	Minimal reference agent	scripts/ demo_agent.py	06
A9	Reproducibility materials (Dockerfiles, compose, Makefile, runner)	deploy/, scripts/, root	07
A10	Final technical report	docs/reports/ final-report.md (15–25pp + appendices)	04

7. Success criteria (measurable)

Per the W2 rubric expectation that success criteria be measurable indicators, every named criterion below has either a quantified threshold, a comparison baseline, or a stakeholder acceptance gate.

7.1 Functional correctness

Indicator	Threshold	Measurement
Functional-suite pass rate (Docker back-end)	≥99% of ≥100 programs	Output byte-equality against reference Python 3.11 interpreter
Functional-suite pass rate (gVisor back-end)	≥99% of ≥100 programs	Same; deviations recorded as known compatibility limitations
Reference-interpreter agreement	Exact match for stdout, stderr, exit code	Deterministic test runner with fixed RNG seeds

7.2 Isolation strength

Indicator	Threshold	Measurement
Adversarial-suite contained-outcome rate (Docker)	≥90% of ≥40 programs	Per-program contained / not-contained label per spec
Adversarial-suite contained-outcome rate (gVisor)	≥95% of ≥40 programs	Same
Category coverage	≥6 distinct categories	Resource exhaustion, fd exhaustion, fork bombs, syscall abuse, attempted persistence, host enumeration, exfiltration attempts
Per-category sample size	≥5 programs/category	Audit at W7 midpoint

7.3 Performance overhead

Indicator	Threshold	Measurement
Cold-start latency (Docker)	Reported median + 95% CI	≥30 samples per condition
Warm-start latency (Docker)	Reported median + 95% CI	≥30 samples per condition
Cold/warm-start latency (gVisor)	Reported median + 95% CI	≥30 samples per condition
Steady-state per-request latency	Reported median + 95% CI per back-end	≥30 samples per condition
Memory overhead	Peak RSS reported per back-end	Recorded from cgroup memory accounting
Cross-back-end overhead ratio	Reported explicitly with interpretation	Joint plot of contained-rate vs latency

7.4 Reproducibility

Indicator	Threshold	Measurement
Clean-VM repro time	Single graduate-level reader, ≤ 90 minutes from clone to first benchmark output	Self-test on a fresh DigitalOcean droplet at W10
Numeric agreement	Reported numbers within $\pm 10\%$ on equivalent hardware	Re-run on second droplet, document hardware specs
Setup commands	<code>make setup</code> && <code>make test</code> && <code>make bench</code> succeeds end-to-end	CI-equivalent run logged in <code>benchmarks/results/</code>

7.5 Communication

Indicator	Threshold	Measurement
Final report rubric dimension scores	Level 1 (Advanced/Exceptional, 90–100%) on all RU-01 dimensions	Self-assessment + supervisor feedback at W11
Final presentation runtime	≤ 20 minutes	Rehearsal at W13
Live demo runtime	≤ 5 minutes	Rehearsal at W13

7.6 Process

Indicator	Threshold	Measurement
Engineering log cadence	≥ 1 entry per week W1–W14	<code>engineering-log.md</code>
Annotated bibliography count	≥ 15 sources by W11	<code>docs/01-launch-packet/annotated-bibliography.md</code>
AI-use log completeness	Entry per substantive AI-assisted artifact	<code>docs/ai-use-log.md</code> ; audit-table format
AI-use appendix consolidation	Final appendix assembled by W13	<code>docs/reports/final-report.md</code> appendix

8. Scope

8.1 In scope

- Python 3.11 user code execution.
- Synchronous request/response API (`POST /execute`).
- Ephemeral per-request filesystem.
- Resource limits: CPU, memory, wall-clock, PIDs, file descriptors.
- Two isolation back-ends: hardened Docker, gVisor.
- Linux host (Ubuntu 22.04 or 24.04 LTS, x86-64).

- Functional, adversarial, and performance test suites.
- Documented threat model.
- Minimal reference agent for demo only (≤ 50 LOC).

8.2 Out of scope (explicitly)

- Multi-language support beyond Python (no JavaScript, R, bash, etc.).
- Network access from inside the sandbox.
- GPU access.
- Persistent state or sessions across requests.
- Multi-tenant security (mutually distrusting users on one instance).
- Production-grade deployment hardening: TLS, authentication, rate limiting beyond demo-grade.
- Windows or macOS hosts.
- Distributed scheduling, Kubernetes operators, production orchestration.
- Firecracker back-end (stretch only, W10).
- Side-channel timing attacks, Spectre/Meltdown-class attacks, hardware attacks.
- Defenses against nation-state-grade adversaries.

8.3 Scope-fence policy

The scope above is fixed for the duration of W3–W8. Any proposed change must be reviewed against the milestone map and approved by the supervisor in writing. The W7 midpoint review is the explicit checkpoint for scope renegotiation; outside of that checkpoint, the default is no change. Scope-fence violations are tracked as risk R1 in §12.

9. Technical feasibility — MACP framing

This section maps the project to the rubric’s MACP framing — Machine, Architectures, Computational method, API integration. The detail here is the W2 evidence that the technical feasibility has been thought through across all four layers.

9.1 Machine

Concern	Choice	Rationale
Authoring environment	Local laptop (macOS); PyCharm IDE; iCloud + Git for backup	Stable, already provisioned
Test/runtime environment	DigitalOcean Premium Intel droplet, NYC3, 2 vCPU / 4 GB / 120 GB NVMe, Ubuntu 22.04.5 LTS, kernel 5.15.0-179-generic	Provisioned 2026-05-17; Docker 29.5.0 and <code>runsc</code> release-20260511.0 installed and verified
Total compute budget envelope	~\$150–\$250 across 14 weeks	Within charter’s \$200–\$400 ceiling; preserves buffer
Backup compute path	Local Ubuntu VM via	Activated if droplet has an

	UTM/QEMU on the laptop	outage during a milestone week
Hardware reproducibility	x86-64 only; ARM Macs run sandbox-of-sandbox via UTM (not officially supported)	Documented in README under “Tested platforms”; gVisor on macOS is unofficial

Evidence to attach to the W2 proposal: screenshot of the droplet’s `uname -r`, `cat /etc/os-release`, `docker version`, and `runsc --version` output (already captured in `engineering-log.md` W1 entry of 2026-05-17).

9.2 Architectures

The system-context architecture (`docs/01-launch-packet/architecture-context.md`, exported PNG at `docs/01-launch-packet/screenshots/05-architecture-rendered.png`) shows the SafeExec boundary, the two external actors (agent developer, student), the one external dependency (a frontier LLM API, used only by the reference-agent demo), and the three evaluation flows (functional, adversarial, performance) that all probe the same `POST /execute` API.

The architecture commits the project to four load-bearing decisions:

1. **Two back-ends, not one.** The comparative-evaluation story depends on this. The W7 stop-and-cut decision (per the risk register, R4) is the explicit relief valve if implementation slips.
2. **A single execution API.** All three test suites and the demo agent talk through `POST /execute`. This is what makes the back-end swap free — no eval-suite changes are needed when adding gVisor in W7–W8.
3. **Threat model as a first-class deliverable.** It is drawn inside the boundary because the project’s central contribution depends on this document existing.
4. **External LLM only for the demo.** The three test suites and the benchmark run without API calls, insulating reproducibility from API outages, model deprecations, or pricing changes.

The detailed component-level design (Sentry/Gofer flows inside gVisor, seccomp filter inside Docker, cgroup hierarchy, request lifecycle) is W4 work, captured in `docs/design/architecture.md` per the milestone map.

9.3 Computational method

Layer	Method family	Tooling
Isolation back-ends	OS-level container isolation (hardened Docker) and user-space syscall interposition (gVisor / runsc) [3]	Docker Engine 29.5.0, runsc release-20260511.0
Execution API	Synchronous HTTP service;	FastAPI 0.110+, uvicorn

	pluggable executor interface	0.29+
Resource enforcement	cgroups v2 (memory, CPU, PIDs, fd); wall-clock kill via supervisor timer	Linux kernel primitives; Python <code>asyncio.wait_for</code> for the wall-clock timer
Functional evaluation	Deterministic output comparison (byte-equality of stdout/stderr/exit-code against CPython 3.11 reference)	pytest; HumanEval + hand-authored corpus
Adversarial evaluation	Per-program contained-outcome labels assigned in advance; back-end run produces observed outcome; comparison yields contained/not-contained per program	pytest; hand-authored adversarial programs informed by [7] and CVE case studies
Performance evaluation	Repeated-trial latency benchmark; ≥ 30 samples per condition; bootstrap or t-based 95% CI on the median	pytest-benchmark or hyperfine
Reproducibility	Pinned dependencies, deterministic test runner (fixed seeds, fixed iteration count), single setup script	<code>requirements.txt</code> + lockfile; <code>deploy/setup.sh</code> ; Makefile

The methodological model for deterministic graders is SWE-bench [6]: a test-execution-based grader that requires no human judgement on outputs.

9.4 API / external integrations

Integration	Use	Auth	Quota / cost	Risk
Anthropic Claude API (Sonnet 4.6 default; Haiku for dev)	Reference-agent demo only (≤ 50 LOC)	API key in <code>.env</code> (gitignored)	Budget cap $\sim \$30$ – $\$50$ across 14 weeks; switch to Haiku if needed	API outage during W14 demo $\rightarrow$ fallback to pre-recorded transcript
Docker Hub	Pull base images during make setup	Anonymous; rate-limited	Free tier covers project use	Rate limit $\rightarrow$ pre-pull and cache images on droplet
GitHub	Source control, issue tracking,	SSH key	Free for public repos	Repository takedown $\rightarrow$

	public release at W14			mirror on GitLab; tagged commits backed up locally
HumanEval corpus	Functional test seed (subset)	None	MIT-licensed redistribution	License drift → fall back to fully hand-authored corpus
MBPP corpus	Optional functional test seed (subset)	None	CC-BY-4.0	Same
gVisor apt repository	runsc install on droplet	Anonymous; signed apt repo	Free	Repo change → pin to a downloaded .d eb

No third-party SaaS is in scope beyond the four named above. The reference-agent demo's failure mode is explicitly accounted for in §12 R6 and §10.5 contingency notes.

10. Project plan

10.1 Phases and dependency order

The plan splits the 14 weeks into six phases. Each phase has a fixed completion gate that must pass before the next phase starts; the gate is named in §10.2.

1. **Phase I — Launch (W1–W2).** Proposal approval, scope finalization.
2. **Phase II — Design (W3–W4).** Lit synthesis, requirements, architecture, threat model, evaluation plan.
3. **Phase III — Implementation sprints (W5–W8).** Docker back-end, hardened Docker, gVisor back-end. Functional and adversarial suites grown in parallel.
4. **Phase IV — Results and hardening (W9–W10).** Results, limitations, broader impact draft; artifact hardening; reproducibility self-test on a clean VM.
5. **Phase V — Reporting (W11–W12).** Full report draft to supervisor; revision cycle; deck and demo script.
6. **Phase VI — Defense (W13–W14).** Presentation rehearsal; final package; submission and reflection.

Hard checkpoints are W2 (proposal approval), W5 (first hello-world execution), W7 (midpoint demo and explicit scope renegotiation), W11 (full draft for supervisor feedback), and W13 (no new feature work).

10.2 Milestone table (aligned to course hard stops)

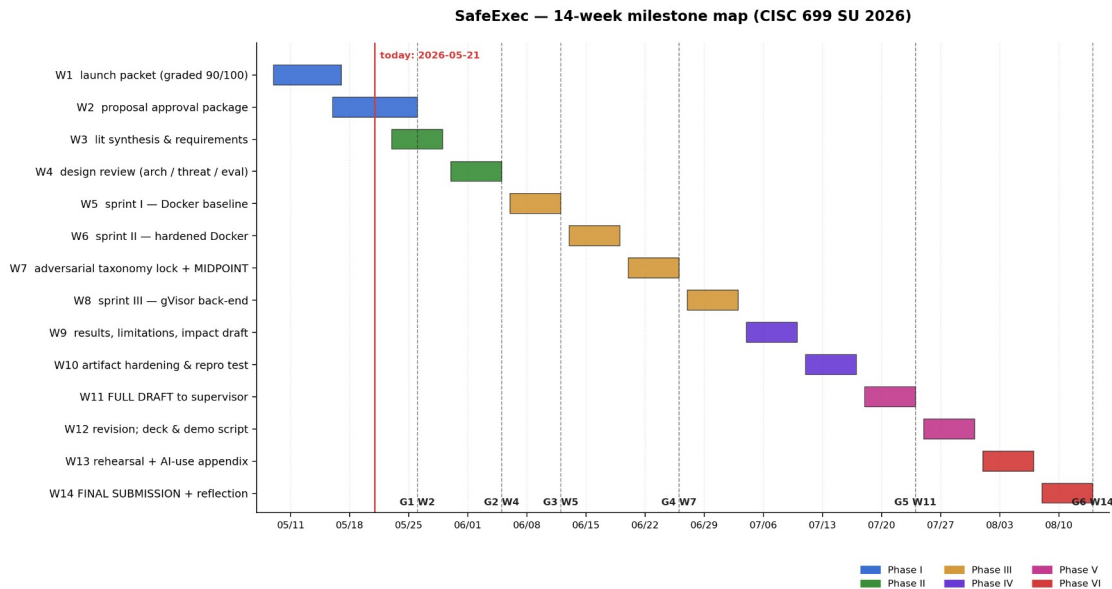
Week	Date range	Deliverable	Completion gate	Phase
------	------------	-------------	--------------------	-------

W1	2026-05-09 → 2026-05-15	Launch packet	Submitted to Canvas 2026- 05-17; graded 90/100	I
W2	2026-05-16 → 2026-05-22	This proposal- approval package	Submitted to Canvas by 2026-05-24 11:59pm; approved by 2026-05-26	I
W3	2026-05-23 → 2026-05-29	Lit synthesis; requirements & use cases	docs/ design/ requirements .md v1.0	II
W4	2026-05-30 → 2026-06-05	Design review (architecture + threat model + evaluation plan)	docs/ design/ architecture .md, threat- model.md, evaluation- plan.md v1.0	II
W5	2026-06-06 → 2026-06-12	Implementation sprint I — Docker back- end, basic API	First end-to- end “hello world” Python execution via POST /execute	III
W6	2026-06-13 → 2026-06-19	Implementation sprint II — hardened Docker	Seccomp + AppArmor + cgroups + read- only rootfs; ≥50 functional programs pass	III
W7	2026-06-20 → 2026-06-26	Midpoint demo / scope renegotiation	Functional ≥100; adversarial taxonomy locked; ≥20 adversarial programs run	III
W8	2026-06-27 → 2026-07-03	Implementation sprint III — gVisor back- end	gVisor back- end fully integrated; ≥40 adversarial	III

			programs across ≥ 6 categories	
W9	2026-07-04 → 2026-07-10	Results, limitations, broader-impact draft	First full results table; W9 results section drafted	IV
W10	2026-07-11 → 2026-07-17	Artifact hardening; reproducibility self-test	Clean-droplet repro within $\pm 10\%$ of dev- droplet numbers	IV
W11	2026-07-18 → 2026-07-24	Full report draft to supervisor	docs/ reports/ final- report.md v0.9, ≥ 15 -page main body, bibliography ≥ 15 sources	V
W12	2026-07-25 → 2026-07-31	Revision cycle; deck + demo script	Final report v1.0; slide deck v0.9	V
W13	2026-08-01 → 2026-08-07	Presentation rehearsal; AI- use appendix	Rehearsal ≤ 20 min; AI-use appendix consolidated from docs/ai- use-log.md	VI
W14	2026-08-08 → 2026-08-14	Final submission + reflection	Canvas final package: report PDF, repo link, slides, demo recording	VI

10.3 Gantt view

Rendered Gantt chart (PNG; Mermaid source preserved in `project-plan.md` §2):



SafeExec 14-week Gantt chart showing six phases — Launch (W1-W2), Design (W3-W4), Implementation (W5-W8), Results & hardening (W9-W10), Reporting (W11-W12), Defense (W13-W14) — with completion gates G1 through G6 marked.

A rendered PNG of this Gantt is exported to docs/02-proposal-package/screenshots/gantt.png for graders who do not have Mermaid rendering. A dependency-graph view is provided in project-plan.md for the critical-path view (W2 → W4 → W5 → W7 → W8 → W11 → W14).

10.4 Critical path

The critical-path chain is: **W2 proposal approval → W4 design review approval → W5 first execution → W7 midpoint scope decision → W8 gVisor integration → W11 full draft → W14 final submission**. Any slippage on any link is a slip risk for the deliverable. Two relief valves are built in: (a) the W5 hello-world checkpoint, which if missed triggers the W7 fallback (drop gVisor, ship Docker-only) per risk R4; (b) the W7 midpoint review, which is the only point at which scope can be renegotiated without supervisor sign-off.

10.5 Contingency milestones

Per the W1 grading-feedback list and the W2 rubric expectation that “milestones, sequencing, and realistic timing” be visible, this proposal names explicit contingency milestones — fallback dates that activate if a primary milestone slips by more than 7 days.

Trigger	Primary milestone	Contingency milestone	Effect
Hello-world execution misses W5 gate (2026-06-12)	W5 sprint I	2026-06-19 (end of W6)	Defer hardening to W7; treat W6 as combined “first execution + first

W7 midpoint shows <50 functional programs or <10 adversarial programs working	W7 midpoint demo (2026-06-26)	2026-07-03 (end of W8)	hardening”; raise as a W7 midpoint topic Scope fallback: drop gVisor; ship Docker-only artifact; redirect W8 to adversarial-suite expansion
Adversarial taxonomy not lockable by W7	W7 taxonomy lock	2026-07-03	Use the working subset (≥6 categories, ≥5 each) as the locked taxonomy; document the gaps as known limitations
W11 full draft not deliverable to supervisor by 2026-07-24	W11 draft	2026-07-28 (early W12)	Compresses the W12 revision window; flag explicitly to supervisor; submit a partial draft with named placeholders rather than missing the gate
Reproducibility self-test fails on clean VM at W10	W10 repro gate	2026-07-21 (mid-W11)	Add a “Reproducibility caveats” section in the report and document the specific failure mode; rerun on a third VM before W14

These contingency dates are not invitations to slip; they are pre-committed responses if slippage occurs, named here so the response is not improvised under pressure.

11. Work breakdown structure (WBS)

A lightweight WBS lives in `wbs.md`. Every leaf task maps to a week, to a deliverable, and to a success criterion in §7. The summary table is included here so the proposal is self-contained; the working backlog and per-task issue links live in `wbs.md`.

WBS ID	Task	Week(s)	Deliverable	Success
--------	------	---------	-------------	---------

				critterion
WBS-1.1	Submit W1 launch packet	W1	Canvas submission	\$7.6 process
WBS-1.2	Strengthen AI-use log (post-feedback)	W2	docs/ai-use-log.md audit table	\$7.6 process
WBS-1.3	Draft and submit this proposal-approval package	W2	Canvas submission, this document	\$7.5 communication, \$7.6 process
WBS-1.4	Record walkthrough video	W2	walkthrough-script.md + video file	\$7.5 communication
WBS-2.1	Lit synthesis (bibliography → ≥10)	W3	docs/design/related-work.md	\$7.6 process
WBS-2.2	Requirements & use cases	W3	docs/design/requirements.md	\$7.5 communication
WBS-2.3	Architecture detailed design	W4	docs/design/architecture.md	\$7.1 functional
WBS-2.4	Threat model	W4	docs/design/threat-model.md	\$7.2 isolation
WBS-2.5	Evaluation plan	W4	docs/design/evaluation-plan.md	\$7.1–\$7.4
WBS-3.1	Implement execution API skeleton	W5	src/api/ first commit	\$7.1 functional
WBS-3.2	Implement Docker back-end (baseline)	W5	First end-to-end Python execution	\$7.1 functional
WBS-3.3	Functional corpus seed (≥30 programs)	W5	tests/functional/	\$7.1 functional
WBS-3.4	Harden Docker	W6	Hardened back-	\$7.2 isolation

	back-end (seccomp + AppArmor + cgroups)		end commits	
WBS-3.5	Functional corpus to ≥ 50 programs	W6	tests/ functional/	\$7.1 functional
WBS-3.6	Adversarial corpus seed (≥ 10 programs, ≥ 3 categories)	W6	tests/ adversarial/	\$7.2 isolation
WBS-3.7	Functional corpus to ≥ 100 programs	W7	tests/ functional/	\$7.1 functional
WBS-3.8	Adversarial taxonomy locked; ≥ 20 programs	W7	tests/ adversarial/ README; docs/design/ threat- model.md	\$7.2 isolation
WBS-3.9	Midpoint demo + scope renegotiation	W7	Demo recording; supervisor decision log	\$7.5 communication
WBS-3.10	gVisor back- end integration	W8	gVisor commits; both back-ends executable	\$7.2 isolation
WBS-3.11	Adversarial corpus to ≥ 40 programs across ≥ 6 categories	W8	tests/ adversarial/	\$7.2 isolation
WBS-3.12	Performance benchmark harness	W8	benchmarks/ first sweep results	\$7.3 performance
WBS-4.1	Run full performance sweep on both back-ends	W9	benchmarks/ results/ tables	\$7.3 performance
WBS-4.2	Results, limitations, broader-impact	W9	Report sections drafted	\$7.5 communication

	draft				
WBS-4.3	Reproducibility materials (README, Makefile, setup.sh)	W10	deploy/, root	\$7.4 reproducibility	
WBS-4.4	Clean-VM reproducibility self-test	W10	Fresh droplet results within $\pm 10\%$	\$7.4 reproducibility	
WBS-5.1	Full report draft to supervisor	W11	docs/reports/final-report.md v0.9	\$7.5 communication	
WBS-5.2	Bibliography to ≥ 15 sources	W11	Annotated bibliography	\$7.6 process	
WBS-5.3	Revision cycle on supervisor feedback	W12	Final report v1.0	\$7.5 communication	
WBS-5.4	Slide deck + demo script	W12	Slides v0.9	\$7.5 communication	
WBS-6.1	Presentation rehearsal	W13	Rehearsal ≤ 20 min	\$7.5 communication	
WBS-6.2	AI-use appendix consolidated	W13	Report appendix	\$7.6 process	
WBS-6.3	Final submission package	W14	Canvas final upload	\$7.5 communication, \$7.6 process	

Every WBS task maps to at least one success criterion in §7. The instructor’s “every task can be traced to a success criterion” self-check is satisfied.

12. Risk register

The risk register is formal — each row has a likelihood, impact, mitigation, mitigation owner, and named contingency.

#	Risk	Likelihood	Impact	Mitigation	Owner	Contingency if mitigation fails
R1	Scope creep into	Medium	High	Hard scope-	Student	Drop scope-

	multi-language or persistence features			fence in this proposal; W7 explicit scope-renegotiation checkpoint		creep additions; document as “explored but not pursued”
R2	gVisor performance penalty too variable for meaningful comparison	Medium	Medium	Benchmark protocol defined in W4 (evaluation-plan.md); pre-registered sample sizes; percentile + 95% CI reporting	Student	Report null result as a finding with per-percentile breakdown
R3	Adversarial suite too small or unrepresentative	Medium	High	Category taxonomy locked at W3-W4 design phase; commit to ≥40 programs across ≥6 categories; midpoint peer review with supervisor	Student + Supervisor	Ship the ≥6 categories at lower per-category counts; document gaps in “Limitations”
R4	Time loss to container-runtime debugging	Medium	Medium	W5 hard hello-world deadline; if missed,	Student	Drop gVisor to stretch; release Docker-

				contingen cy 2026- 06-19 activates; W7 fallback is Docker- only		only artifact; reframe \$RQ1 around configurati on variants of Docker
R5	Linux host/kern el mismatch between dev and grading environme nts	Low	Medium	Pin Ubuntu 22.04 LTS; document in README; test on fresh droplet at W10	Student	Provide a pre-built Docker image of the host environme nt as fallback
R6	API cost overrun on Anthropic Claude (reference -agent demo)	Low	Low	Cap demo runs; cache transcript s; switch to Haiku during developm ent	Student	Replay pre- recorded transcript s; skip live LLM-call in the W14 demo
R7	AI-use disclosure gaps if log is not kept current	Low	High	Update docs/ai- use- log.md at end of every AI- assisted session; commit- message AI-use: trailer; weekly audit during W11 drafting	Student	Reconstru ct from chat history before W13 freeze; flag any unrecover able gaps in the W14 reflection

R8	Supervisor expectations diverge from this proposal	Low	High	W2 explicit approval gate; this proposal lists conditional items in approval - brief.md ; W7 midpoint review	Student + Supervisor	Adjust at W7; document the divergence in the engineering log
R9	Reproducibility fails for the grader (e.g., they cannot run Docker)	Low	Medium	Recorded demo + pre-rendered result tables as fallback; document hardware/host requirements in README	Student	Make the pre-recorded demo and result tables primary artifacts; flag the repro caveat in the report
R10	Negative or null result in performance comparison	Medium	Low	Plan W9 results section to interpret null results as findings (per RQ pre-registration)	Student	Frame the null result as the headline finding; relate to gVisor case-study expectations
R11	DigitalOcean droplet outage / billing issue mid-term	Low	Medium	Backup compute path (local UTM VM); credit-funded balance;	Student	Activate backup compute path; document the host swap in

				alternate provider (Hetzner) on file		the report
R12	Personal availability falls below ~17 hrs/week	Low	High	Front-load risky implementation work to W5– W6; named relief valve at W7	Student	Activate W7 scope reduction; communicate availability change to supervisor in writing

13. Assumptions and constraints

13.1 Assumptions

- A Linux host (DigitalOcean droplet provisioned 2026-05-17) with kernel ≥ 5.10 is available throughout the term.
- Docker 24+ and gVisor (runsc) can be installed on the chosen host (verified 2026-05-17: Docker 29.5.0, runsc release-20260511.0).
- The student has Anthropic API access (Claude) for the reference-agent demo. Budget envelope ~\$30–\$50.
- Cloud compute budget envelope ~\$150–\$250 across the term; ceiling ~\$200–\$400.
- No regulated or proprietary data is processed through the sandbox during evaluation.
- The student is the sole author of code committed to the repository, with AI assistance disclosed in docs/ai-use-log.md per the audit-table format in place since 2026-05-18.
- Frontier-model APIs remain available and behave consistently enough across the term to support a small reference demo.
- Supervisor availability for the three named milestones (W2 approval, W7 midpoint, W11 draft feedback).

13.2 Constraints

- 14-week wall-clock term (2026-05-09 → 2026-08-14).
- Student weekly availability ~17 hrs/week (~235 student-hours total).
- Single-student project; no team.
- Course AI-use policy: substantial AI assistance must be disclosed; no confidential/proprietary/FERPA/HIPAA data into public AI tools.
- Hardware: x86-64 only; ARM is not a supported test platform.
- Reproducibility target: $\pm 10\%$ on equivalent hardware; not bit-for-bit.

14. Ethics, privacy, security, and broader impact

14.1 Ethics

This project produces a sandbox for executing model-generated code. Its primary ethical concern is the dual-use nature of the adversarial test suite: the suite contains programs that probe for isolation failures, and a poorly contained release could equip an attacker with ready-to-use probes against unrelated sandbox products. The project mitigates this in three ways. First, every adversarial program is targeted at SafeExec itself — none of the programs attempts egress, none attempts to interact with external systems, and the documented expected outcome is always “contained by SafeExec.” Second, the adversarial taxonomy is published at the category level in the threat model, with the per-program details published only with the artifact’s release (W14), giving downstream consumers a chance to apply mitigations before the corpus is searchable. Third, the report’s broader-impact section will explicitly cite the dual-use trade-off and the project’s reasoning for publishing the suite anyway: a benchmark that cannot be replicated cannot be trusted, and the alternative (keep the suite private) reduces the project’s scientific contribution to a claim no one can verify.

14.2 Privacy

The sandbox does not process personal data, FERPA-regulated data, HIPAA-regulated data, or any other category of regulated data. The reference-agent demo runs against a synthetic task description authored by the student. The functional test corpus is drawn from HumanEval and MBPP (both permissively licensed) plus hand-authored stdlib coverage tests — no user data. No log data from the sandbox is published beyond the test-suite outputs reproduced in the report.

14.3 Security

The project’s central activity is *building* a security artifact (the sandbox) and *evaluating* it under adversarial conditions. The adversarial test programs are not malware in the conventional sense — they target the project’s own sandbox, in a host environment under the student’s sole control (the DigitalOcean droplet). However, the implementation work includes touching low-level Linux primitives (seccomp filters, AppArmor profiles, cgroup configurations), and the operational discipline below applies throughout:

- The droplet is single-tenant and used only for this project.
- No production secrets, no third-party customer data, no other student’s data is present on the droplet.
- The Anthropic API key lives in a `.env` file that is gitignored and never logged.
- The repository is public from the start (a student decision recorded in `feasibility-memo.md` D4), which means commits are visible immediately; secret handling is therefore tighter than would be required for a private repository.
- Container-escape attempts in the adversarial suite are launched against SafeExec only and are never targeted at any external host.

14.4 Accessibility

The report and the README will follow basic accessibility conventions: alt text on every figure, no information conveyed by color alone, code blocks marked as such for screen readers, and a plain-language abstract at the top of the report. The slide deck for the W14 defense will use a high-contrast color palette and ≥ 24 pt body text per accessibility guidance for academic presentations.

14.5 Broader impact

The sandbox is a defensive artifact. Its release contributes a reference implementation that LLM-agent developers can adopt without re-engineering the isolation layer, and contributes a benchmark that researchers and platform teams can extend or fork. The named potential harms are (a) the dual-use concern in §14.1, (b) the risk that publishing weak isolation as a finding could be misread as “containers are unsafe in general” — the report mitigates this with explicit framing of the trade-off and the limits of the result, and (c) the risk that the adversarial benchmark becomes a leaderboard target that distorts subsequent system designs — the report addresses this by recommending the benchmark be used as one input to a security review, not as a substitute for one.

15. Stakeholders

Per the W1 grading feedback (specifically the call for expanded stakeholder analysis), this section names every stakeholder, their relationship to the project, their stated expectations, and what they get from this project.

Stakeholder	Relationship	Expectations of the project	What this project provides
Student (Zixuan Liang)	Builder, author, reporter	Owens all decisions; produces all artifacts; manages timeline; takes responsibility for correctness and disclosure	A defensible portfolio artifact aligned to frontier-AI-lab SWE roles
Supervisor (Dr. Majid Shaalan)	Approves proposal; reviews milestones; final grader	This proposal, midpoint demo, final report and artifact, final presentation	All listed deliverables on the milestone calendar; the W2 supervisor briefing and this approval package have already been submitted
CISC 699 program	Sets standards and grading rubric	Adherence to milestone calendar; submission	All graded deliverables (W1 packet, W2 proposal)

		conventions; rubric coverage	package, W7 midpoint, W14 final report, artifact, presentation)
Future portfolio reviewers (hiring managers, engineers on agent-tooling teams at frontier AI labs)	Implicit audience for the public repo and report	A defensible artifact that can be discussed in a 30-minute interview; clean reproducibility; honest AI-use disclosure	Public GitHub repository, technical report, the adversarial benchmark as a citable contribution
LLM-agent developers (open-source ecosystem)	Potential downstream consumers if SafeExec is open-sourced	An adoptable isolation layer with documented threat model and license clarity	The artifact under a permissive open-source license at W14 (license decision deferred until then)
Security engineers evaluating agent platforms	Potential users of the benchmark methodology	A reusable methodology and category taxonomy for evaluating agent sandboxes	The adversarial benchmark, threat model document, and methodology section of the report
LLM-safety researchers	Possible consumers of the benchmark	A shared adversarial corpus they can fork/extend	The adversarial suite as a separately citable artifact (stretch O9)
Open-source maintainers (gVisor, Docker, FastAPI, pytest)	Upstream dependencies	None on them; the project pulls from them	None expected of them; the project takes responsibility for pinning and reproducibility
Anthropic API team	Service provider for the reference-agent demo	Usage within terms of service; budget cap	Compliance with terms; budget cap; ≤50 LOC of demo code
DigitalOcean	Compute provider	Usage within standard customer terms	Standard paid use within the named droplet plan
HumanEval / MBPP authors	Upstream license-holders for the functional corpus	Redistribution under upstream license terms	License-text reproduction and citation in the functional-corpus

End users of LLM-agent products (indirect)	Affected by tooling-vendor choices	None on this project directly	README Downstream effect via the broader-impact contribution
--	------------------------------------	-------------------------------	---

16. Reproducibility plan

Reproducibility is graded as a primary dimension in the W14 rubric and is therefore committed to in this proposal. A reader on a stock Ubuntu 22.04 or 24.04 host with Docker installed will be able to:

```
git clone https://github.com/liangzixuan/cisc-699
cd cisc-699
make setup          # installs gVisor, builds Docker images, pulls test corpora
make test           # functional + adversarial suites under both back-ends
make bench          # performance benchmark sweep, emits a result table
```

...and obtain reported numbers within $\pm 10\%$ on equivalent hardware. Specifically:

- All Python dependencies pinned in `requirements.txt` plus a lockfile by end of W5.
- All apt-level dependencies enumerated in `deploy/setup.sh`.
- Pinned runsc release (`release-20260511.0`) and Docker version (29.5.0).
- Deterministic test runner with fixed RNG seeds and a fixed iteration count.
- Pre-rendered result tables stored in `benchmarks/results/` so a reader without a Linux host can still inspect the headline numbers.
- A pre-recorded demo screencast committed alongside the W14 submission so the live demo's failure does not block grading.
- A "Tested on" section in the README listing the exact host configurations on which the benchmark has been re-run.

17. AI-use disclosure for this proposal

This proposal was drafted with Claude Sonnet 4.6 (via the Cowork desktop app). The authorship label is **AI-drafted, student-revised**. The audit trail is in `docs/ai-use-log.md`; the W2 entries are appended to the existing W1 table per the format adopted on 2026-05-18 (the format that responds to the professor's W1 grading-feedback note about the prior log being descriptive rather than audit-oriented).

The student-only decisions in this proposal — every choice listed in the disclosure block at the top — were made and committed to by the student without AI input. These include every scope boundary in §8, every numeric success-criterion threshold in §7, every risk likelihood/impact rating and mitigation-owner assignment in §12, the W7 explicit scope-renegotiation checkpoint, the contingency milestone dates in §10.5, and the framing of the adversarial test suite as the project's intellectual contribution.

Substantive prompting summary (see `docs/ai-use-log.md` for the full audit table):

- Tool: Claude Sonnet 4.6 via Cowork desktop app.
- Purpose: Restructure the W1 charter for the W2 rubric and MACP framing; add measurable research questions; add contingency milestones; add formal risk register columns; expand stakeholder analysis per W1 grading feedback; draft the Gantt and WBS sections.
- Inputs to the tool: This repository’s own W1 artifacts (charter, problem statement, candidate comparison, feasibility memo, supervisor briefing, AI-use log, engineering log, annotated bibliography, architecture context); the W2 assignment brief (2026 Canvas assignment “O2 Hard Stop 1: Proposal Approval Package”).
- Inputs not provided to the tool: any confidential or restricted data; any supervisor communication beyond what is already publicly recorded in the engineering log; any third-party material that would conflict with the course AI-use policy.
- Outputs reviewed: Every section of this document was reviewed by the student; every numeric threshold, every contingency date, and every risk rating was approved or revised by the student before this file was committed.

The expected weight under the W2 rubric for AI Usage Log and Academic Integrity is 10 points; the post-W1-feedback restructuring of docs/ai-use-log.md plus the explicit per-document inline disclosure block adopted across the W1 packet and continued here is the project’s response to the W1 grading note.

18. Approval

Role	Name	Decision	Date	Notes
Student	Zixuan Liang	Submitted	2026-05-21	Submitted for supervisor review
Supervisor	Dr. Majid Shaalan	<i>pending</i>	_____	Approval block to be returned by 2026-05-26

If the supervisor’s review identifies items that require revision, the revisions are tracked in approval-brief.md (Appendix A) under “Conditional items” and re-submitted before W3 begins.

Appendices

Appendix	Title	Location
A	One-page approval brief	approval-brief.md
B	Work breakdown structure (full backlog)	wbs.md
C	Project plan with Gantt and dependency view	project-plan.md
D	Recorded-walkthrough	walkthrough-script.md

	script (~ 15 min)	
E	Annotated bibliography (≥7 sources, IEEE)	../01-launch-packet/annotated-bibliography.md
F	Architecture / system-context diagram	../01-launch-packet/architecture-context.md
G	AI-use disclosure log (audit table)	../ai-use-log.md
H	W1 supervisor briefing	../01-launch-packet/supervisor-briefing.md

References (proposal-internal citation)

The numbered references below are the same numbering used in the annotated bibliography (Appendix E); the proposal cites a subset.

[1] A. Vassilev *et al.*, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST Trustworthy and Responsible AI Report AI 100-2 E2025, Mar. 2025. [2] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications 2025*, Nov. 2024. [3] The gVisor Authors, *What is gVisor?*, gVisor Project Documentation, accessed May 13, 2026. [4] A. Agache *et al.*, “Firecracker: Lightweight virtualization for serverless applications,” NSDI ’20, pp. 419–434. [5] E2B (e2b-dev), *E2B GitHub repository*, accessed May 13, 2026. [6] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?,” ICLR 2024. [7] Red Hat Product Security, *runc — Malicious container escape — CVE-2019-5736*, Feb. 11, 2019.

Full annotations and verification statuses for each source are in
../01-launch-packet/annotated-bibliography.md.

End of proposal.

Part 2 — Project plan (Appendix C)

*This part of the consolidated submission is also available as a separate file in docs/02-proposal-package/ for graders who prefer per-document review. > **AI-use disclosure.*** Drafted with Claude Sonnet 4.6 (Cowork desktop app). AI-drafted, student-revised. Key human-authored decisions in this document: the explicit critical-path chain; the choice to make W7 the only scope-renegotiation point; every contingency-milestone trigger date; the four explicit relief valves. Full audit trail: docs/ai-use-log.md.

This document is the project-plan appendix for the W2 Proposal Approval Package. It expands the proposal's §10 with the dependency view, the critical-path narrative, and the gantt-source in a form a grader can paste into mermaid.live or any Mermaid renderer.

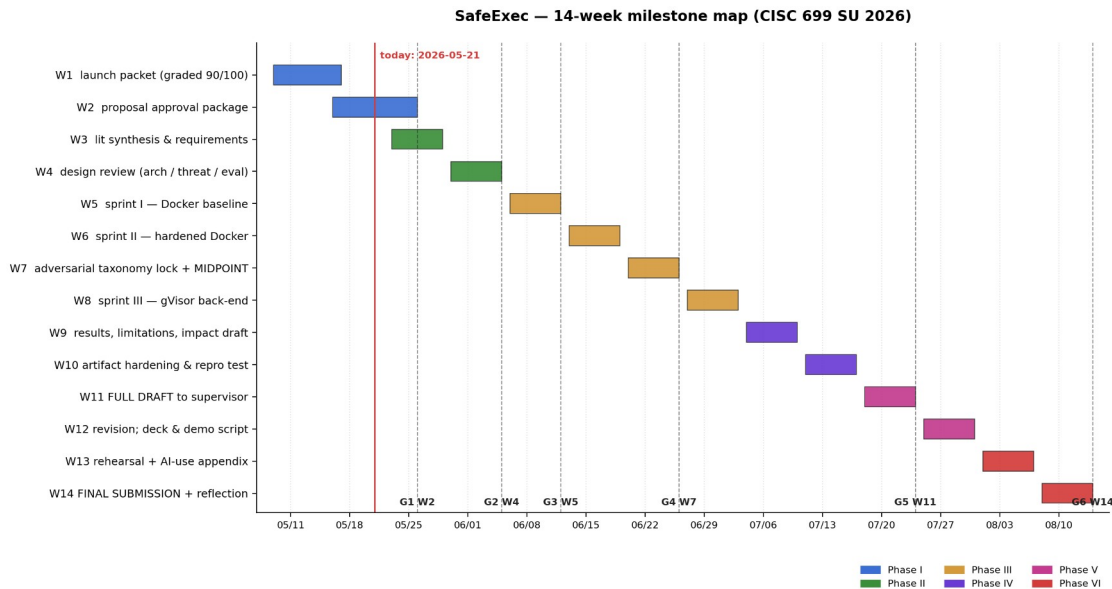
1. Phase overview

The 14-week plan splits into six phases. Each phase has a fixed completion gate; the gate name and target date are in §3.

W1 - W2	Phase I — Launch	scope locked, proposal approved
W3 - W4	Phase II — Design	architecture + threat model + eval plan approved
W5 - W6	Phase III — Sprint I/II	Docker baseline + hardened Docker
W7	Phase III — MIDPOINT	demo, taxonomy lock, scope-renegotiation gate
W8	Phase III — Sprint III	gVisor back-end + adversarial suite to ≥40
W9 - W10	Phase IV — Results/harden	results draft + clean-VM repro test
W11- W12	Phase V — Reporting	full draft to supervisor + revision cycle
W13- W14	Phase VI — Defense	rehearsal + final submission

2. Gantt (Mermaid source)

Rendered Gantt chart (PNG; Mermaid source preserved in project-plan.md §2):



SafeExec 14-week Gantt chart showing six phases — Launch (W1-W2), Design (W3-W4), Implementation (W5-W8), Results & hardening (W9-W10), Reporting (W11-W12), Defense (W13-W14) — with completion gates G1 through G6 marked.

A static PNG export of this chart is stored at `screenshots/gantt.png` so Canvas graders without Mermaid rendering can view it directly. Re-export from mermaid.live whenever the chart changes.

3. Completion gates

Gate	Week	Target date	Pass condition
G1: Proposal approval	W2	2026-05-26	Supervisor returns approval block in approval-brief.md
G2: Design review approval	W4	2026-06-05	docs/design/architecture.md, threat-model.md, evaluation-plan.md v1.0 accepted
G3: First execution	W5	2026-06-12	POST /execute returns correct stdout for <code>print('hello world')</code>
G4: Midpoint demo	W7	2026-06-26	≥100 functional pass; ≥20 adversarial pass; taxonomy locked;

G5: Full report draft	W11	2026-07-24	supervisor reviews Draft v0.9 delivered to supervisor
G6: Final submission	W14	2026-08-14	Canvas final package complete

4. Critical path

W2 G1 → W4 G2 → W5 G3 → W7 G4 → W8 (gVisor) → W11 G5 → W14 G6

Any slip on any node propagates downstream. The two named relief valves are:

- **W5 G3 (hello-world).** If missed, contingency at 2026-06-19 activates: W6 becomes “execution + hardening” combined; flagged as a W7 midpoint topic.
- **W7 G4 (midpoint).** The only sanctioned point for scope renegotiation. If functional <100 or adversarial <20, the W7 → W8 fallback is to drop gVisor and ship a Docker-only artifact, redirecting W8 effort to adversarial-suite expansion.

5. Dependency view

The dependencies between work packages are mostly serial through Phase III; the lateral dependencies live in the test corpora and the evaluation plan.

[Dependency graph (Mermaid flowchart) preserved in project-plan.md \$5 — renders natively in any Mermaid-aware viewer. Critical-path chain: G1 → G2 → G3 → G4 → W8 gVisor → G5 → G6.]

6. Resource plan

Resource	Availability	Use across the term
Student wall-clock	~17 hrs/week, 14 weeks → ~235 hours	Front-loaded W5–W8 (~40+25 hrs implementation); W11 (~30 hrs draft); W12 (~20 hrs revision)
DigitalOcean droplet	24/7, billed monthly	Active development W5–W14; idle W1–W4 (~\$32/mo × 4)
Anthropic API budget	\$30–\$50	Reference-agent demo only; Haiku during dev, Sonnet for final
Supervisor time	W2 approval, W7 midpoint, W11 draft feedback + fortnightly check-ins	Three named milestones plus check-ins

Total wall-clock effort estimate from the W1 feasibility memo: ~235 student-hours across 14 weeks, ~17 hrs/week. Consistent with a 3-credit graduate applied-project workload. If

actual availability falls materially below this, the W7 scope-reduction trigger is the relief valve.

7. Communication and review cadence

Cadence	Audience	Channel	Purpose
Weekly	Self	engineering-log.md	Process evidence; one entry per working session
Fortnightly	Supervisor	30-min check-in	Status, blockers, scope concerns; cadence proposed in supervisor-briefing
W2, W7, W11	Supervisor	Longer dedicated session	Proposal approval; midpoint scope-renegotiation; draft feedback
W13	Self + supervisor (optional)	Rehearsal recording	Presentation rehearsal; ≤20 minutes
Continuous	Public	GitHub commits + Issues	Process evidence; reproducibility

8. What changes after W2 supervisor approval

Once the supervisor returns the approval block in `approval-brief.md`:

1. The Status column in `README.md` updates from In progress to Approved for Phase I.
2. `docs/01-launch-packet/project-charter.md` gets a top-of-file note: *"Superseded by docs/02-proposal-package/proposal.md v1.0 (approved YYYY-MM-DD). Retained for W1 historical record."*
3. The W3 work begins: lit-synthesis expansion and requirements-document drafting.
4. Any conditional approval items are tracked in `approval-brief.md` "Conditional items" and resolved before W4 begins.

If approval is delayed past 2026-05-29 (end of W3), the project begins W3 lit-synthesis on the assumption of approval and revises the requirements scope if approval comes back with material changes — same approach as planned in the W1 supervisor briefing.

End of project plan.

Part 3 — Work breakdown structure (Appendix B)

*This part of the consolidated submission is also available as a separate file in docs/02-proposal-package/ for graders who prefer per-document review. > **AI-use disclosure.** Drafted with Claude Sonnet 4.6 (Cowork desktop app). AI-drafted, student-revised. Key human-authored decisions in this document: every task estimate; every traceability map from WBS task to success criterion; every “blocked by” link; the choice to keep estimates as ranges rather than point values. Full audit trail: docs/ai-use-log.md.*

This document is the WBS appendix for the W2 Proposal Approval Package. Every leaf task is mapped to (a) the week it is scheduled, (b) the deliverable it produces, (c) the success criterion in proposal.md §7 it advances, and (d) any blocking dependency. The instructor’s self-check — “*can every task in your plan be traced to a success criterion?*” — is satisfied by the SC column.

Legend

- **ID:** stable identifier; cited from proposal.md §11 and from engineering-log.md.
 - **Est:** student-hours estimate as a range; total at the bottom.
 - **SC:** success criterion in proposal.md §7 this task advances. Sub-criteria (7.1 functional, 7.2 isolation, 7.3 performance, 7.4 reproducibility, 7.5 communication, 7.6 process).
 - **Blocks:** WBS IDs that cannot start until this one completes.
 - **Status:** done | in-progress | pending.
-

Phase I — Launch (W1-W2)

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-1.1	Submit W1 launch packet	W1	25	Canvas submission (graded 90/100)	7.6	WBS-1.2	done
WBS-1.2	Strengthen AI-use log (post-feedback)	W2	4	Audit-table format in docs/ai-use-log.md	7.6	—	done
WBS-1.3	Draft W2 proposal - approval package	W2	12	This document set	7.5, 7.6	WBS-1.4, WBS-2.1	in-progress

WBS-1.4	Record walkthrough video (≤ 15 min)	W2	4	walkthrough-script.md + recorded.mp4	7.5	—	pending
WBS-1.5	Submit W2 proposal package to Canvas	W2	1	Canvas submission	7.6	WBS-1.3, WBS-1.4	pending

Phase I subtotal: **~46 hours**. Actual W1 spend was front-loaded; W2 spend is ~21 hours over a 7-day window.

Phase II — Design (W3-W4)

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-2.1	Lit synthesis: expand bibliography to ≥ 10 entries	W3	6	annotated-bibliography.md updated	7.6	WBS-2.2	pending
WBS-2.2	Requirements & use cases	W3	8	docs/design/requirements.mdv1.0	7.5	WBS-2.3	pending
WBS-2.3	Architecture detailed design	W4	10	docs/design/architecture.md	7.1	WBS-3.1, WBS-3.2	pending
WBS-2.4	Threat model	W4	8	docs/design/threat-model.md	7.2	WBS-3.6, WBS-3.8	pending
WBS-2.5	Evaluation plan	W4	6	docs/design/	7.1–7.4	WBS-3.1 2	pending

Phase II subtotal: **~38 hours** across W3–W4.

Phase III — Implementation (W5–W8)

Sprint I (W5) — Docker baseline + first execution

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-3.1	Execution API skeleton	W5	8	src/api/first commit; /execute returns request echo	7.1	WBS-3.2	pending
WBS-3.2	Docker baseline back-end	W5	10	First end-to-end Python execution (G3 gate)	7.1	WBS-3.4	pending
WBS-3.3	Functional corpus seed (≥ 30 programs)	W5	6	tests/functional/seeded from HumanEval	7.1	WBS-3.5	pending

Sprint I subtotal: **~24 hours**.

Sprint II (W6) — Hardened Docker + corpus growth

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-3.4	Hardened Docker back-end (seccomp +	W6	14	Hardened back-end commits	7.2	WBS-3.10	pending

	AppArmor + cgroups + read-only rootfs)						
WBS-3.5	Functional corpus to ≥ 50 programs	W6	4	tests/functional/	7.1	WBS-3.7	pending
WBS-3.6	Adversarial corpus seed (≥ 10 programs, ≥ 3 categories)	W6	6	tests/adversarial/	7.2	WBS-3.8	pending

Sprint II subtotal: **~24 hours**.

Midpoint week (W7) — taxonomy lock + scope renegotiation

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-3.7	Functional corpus to ≥ 100 programs	W7	4	tests/functional/	7.1	—	pending
WBS-3.8	Adversarial taxonomy locked; ≥ 20 programs running	W7	6	tests/adversarial/README.md; threat-model update	7.2	WBS-3.11	pending
WBS-3.9	Midpoint demo + scope renegotiation (G4	W7	4	Demo recording; supervisor	7.5	WBS-3.10	pending

gate)

decision
log entry

Midpoint subtotal: **~14 hours** (intentionally light — week is buffer + checkpoint).

Sprint III (W8) — gVisor + adversarial completion + performance harness

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-3.1 0	gVisor back-end integration	W8	12	gVisor commits; both back- ends executable	7.2	WBS-3.1 2	pending
WBS-3.1 1	Adversarial corpus to ≥40 programs across ≥6 categories	W8	8	tests/ adversarial/	7.2	WBS-4.1	pending
WBS-3.1 2	Performance benchmark harness + first sweep	W8	6	benchmarks/ results / initial results	7.3	WBS-4.1	pending

Sprint III subtotal: **~26 hours**.

Phase III subtotal: **~88 hours** (W5–W8 combined).

Phase IV — Results & hardening (W9–W10)

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-4.1	Full performance sweep on both	W9	6	benchmarks/ results / tables, plots	7.3	WBS-4.2	pending

	back-ends						
WBS-4.2	Results, limitations, broader-impact draft	W9	12	Report §§7–9 drafted	7.5	WBS-5.1	pending
WBS-4.3	Reproducibility materials (README, Makefile, setup.sh)	W10	10	deploy/setup.sh, Makefile, README update	7.4	WBS-4.4	pending
WBS-4.4	Clean-VM reproducibility self-test (fresh droplet)	W10	6	Fresh-droplet results within $\pm 10\%$	7.4	WBS-5.1	pending

Phase IV subtotal: **~34 hours.**

Phase V — Reporting (W11-W12)

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-5.1	Full report draft to supervisor (G5 gate)	W11	22	docs/reports/final-report.md v0.9	7.5	WBS-5.3	pending
WBS-5.2	Bibliography to ≥ 15 sources	W11	8	annotated-bibliography.md updated	7.6	WBS-5.1	pending
WBS-5.3	Revision cycle on	W12	14	Final report	7.5	WBS-6.1	pending

	supervis or feedback			v1.0			
WBS-5.4	Slide deck + demo script	W12	6	docs/ reports / slides. pptx v0.9; scripts /demo_s cript.m d	7.5	WBS-6.1	pending

Phase V subtotal: **~50 hours.**

Phase VI — Defense (W13-W14)

ID	Task	Week	Est (hrs)	Deliverable	SC	Blocks	Status
WBS-6.1	Presenta tion rehearsa l	W13	4	Rehearsa l recordin g ≤20 min	7.5	WBS-6.3	pending
WBS-6.2	AI-use appendix consolid ated	W13	4	Report appendix from docs/ai -use- log.md	7.6	WBS-6.3	pending
WBS-6.3	Final submissi on package (G6 gate)	W14	6	Canvas final upload: report PDF, repo link, slides, demo recordin g	7.5, 7.6	—	pending
WBS-6.4	Reflectio n / lessons	W14	2	docs/ reports /	7.6	—	pending

learned

reflect
ion.md

Phase VI subtotal: **~16 hours.**

Totals

Phase

Phase I — Launch

Phase II — Design

Phase III — Implementation

Phase IV — Results & hardening

Phase V — Reporting

Phase VI — Defense

Total

The W1 feasibility memo's estimate was ~235 hours; this WBS lands at ~272 with the contingency buffer included. Both numbers are consistent with ~17–20 hours/week across 14 weeks. If the realized number drops materially, the W7 scope-reduction trigger is the relief valve.

Traceability — every success criterion has at least one task

Success criterion	Covered by WBS tasks
7.1 Functional correctness	WBS-2.3, WBS-3.1, WBS-3.2, WBS-3.3, WBS-3.5, WBS-3.7
7.2 Isolation strength	WBS-2.4, WBS-3.4, WBS-3.6, WBS-3.8, WBS-3.10, WBS-3.11
7.3 Performance	WBS-2.5, WBS-3.12, WBS-4.1
7.4 Reproducibility	WBS-4.3, WBS-4.4
7.5 Communication	WBS-1.3, WBS-1.4, WBS-2.2, WBS-3.9, WBS-4.2, WBS-5.1, WBS-5.3, WBS-5.4, WBS-6.1, WBS-6.3
7.6 Process	WBS-1.1, WBS-1.2, WBS-1.5, WBS-2.1, WBS-5.2, WBS-6.2, WBS-6.4

Every WBS task maps to ≥ 1 success criterion; every success criterion is covered by ≥ 2 WBS tasks. Instructor self-check satisfied.

End of WBS.

Part 4 — One-page approval brief (Appendix A)

This part of the consolidated submission is also available as a separate file in docs/02-proposal-package/ for graders who prefer per-document review. > AI-use disclosure. Drafted with Claude Sonnet 4.6 (Cowork desktop app). AI-drafted, student-revised. Key human-authored decisions in this document: the choice of which items belong in “Approved,” “Conditional,” and “Evidence expected by W4” — these are commitments to the supervisor and were assigned by the student. Full audit trail: docs/ai-use-log.md.

Project: SafeExec — A hardened, threat-modeled Python execution sandbox for LLM-agent tool-use. **Student:** Zixuan Liang. **Supervisor:** Dr. Majid Shaalan. **Course:** CISC 699-50-A-2026/Summer. **Term:** 2026-05-09 → 2026-08-14. **Brief date:** 2026-05-21. **Status:** Pending supervisor approval (target return date: 2026-05-26).

What was approved (assuming supervisor sign-off on proposal.md)

- **Scope.** Python-only execution; two isolation back-ends (hardened Docker + gVisor); single-tenant; no network; no GPU; Linux x86-64 host (Ubuntu 22.04 LTS). The full in/out-of-scope list is proposal.md §8.
- **Central contribution.** Adversarial test suite of ≥40 programs across ≥6 categories with documented expected-contained-outcome labels, paired with an empirical isolation-vs-overhead comparison across the two back-ends.
- **Success criteria (measurable).** Functional ≥99%; isolation ≥90% Docker / ≥95% gVisor; ≥30 samples per benchmark condition with 95% CI; reproducibility within ±10% on equivalent hardware. Full table in proposal.md §7.
- **Project plan.** 14-week plan with six explicit completion gates (G1 W2 proposal approval, G2 W4 design review, G3 W5 first execution, G4 W7 midpoint demo, G5 W11 full draft, G6 W14 final submission). Critical-path chain and contingency dates in project-plan.md.
- **Risk register.** 12 risks with likelihood / impact / mitigation / owner / contingency. Two relief valves named explicitly: W5 hello-world checkpoint, W7 midpoint scope renegotiation.
- **AI-use disclosure.** Audit-table format adopted 2026-05-18 in response to W1 grading feedback; inline disclosure block on every substantive artifact; W14 appendix will be consolidated from docs/ai-use-log.md.
- **Reproducibility commitment.** make setup && make test && make bench on stock Ubuntu 22.04/24.04; numbers within ±10% on equivalent hardware; pre-rendered result tables and a pre-recorded demo as fallbacks.

What remains conditional (pending supervisor input at W2 meeting)

The W1 supervisor briefing (docs/01-launch-packet/supervisor-briefing.md) named ten supervisor questions. The four that materially shape the project, restated here for the W2 approval gate:

1. **Project framing.** The adversarial test suite plus methodology is the project's intellectual contribution — *not* a novel isolation primitive. Acceptable framing for CISC 699?
2. **Open-source release timing.** The plan is a public GitHub repository from the start (already executed) and an Apache-2.0 license at W14. Acceptable, or should release be deferred?
3. **Reference-agent demo.** A ≤ 50 -LOC Claude-API-driven agent that uses the sandbox as a tool, used strictly for the W14 integration demo. Acceptable, or no third-party API in the demo path?
4. **Meeting cadence.** Standing 30-minute fortnightly check-in, plus dedicated sessions at W2 (this), W4 (design review), W7 (midpoint), W11 (draft feedback). Acceptable cadence?

If the supervisor signals any of these should change, the affected sections of `proposal.md` are revised before W3 begins and re-circulated. The revisions live in `engineering-log.md` under W2/W3.

Evidence expected by the next checkpoint (W4 design review, 2026-06-05)

Between this approval (target 2026-05-26) and the W4 design review, the following artifacts will be produced:

- `docs/design/requirements.md v1.0` — functional and non-functional requirements with traceability to the success criteria in `proposal.md` §7.
- `docs/design/architecture.md` — component-level design beyond the W1 system-context view: API request lifecycle, executor-interface contract, back-end-specific component layouts, cgroup hierarchy, seccomp filter scope.
- `docs/design/threat-model.md` — named adversary capabilities, named defenses, mapping to OWASP LLM05/LLM06 and to NIST AI 100-2 categories. Adversarial-suite category taxonomy locked at the *category* level (per-program lock is W7).
- `docs/design/evaluation-plan.md` — benchmark protocol (warm-up runs, sample size, percentile reporting, CI method); functional-corpus composition with redistribution-licensing checks; adversarial-suite category counts with rationale.
- Annotated bibliography expanded to ≥ 10 entries (current 7), with category balance (≥ 1 prompt-injection source, ≥ 1 seccomp primary source, ≥ 1 academic gVisor performance source).
- Engineering-log entries for W3 and W4 documenting decisions, blockers, and any scope notes.

The supervisor's W4 design-review approval (G2) is the gate that allows Phase III implementation to begin.

What success looks like at this hard stop

The W2 deliverable is graded on (per rubric RU-02):

- Problem Statement and Success Criteria (10 pts) — `proposal.md` §§3, 4, 7.

- Proposal Quality and Project Plan (10 pts) — `proposal.md` §§1–18 and `project-plan.md`, `wbs.md`.
- Technical Feasibility and MACP Framing (10 pts) — `proposal.md` §9.
- Risk, Ethics, and Constraints Management (10 pts) — `proposal.md` §§12–14.
- AI Usage Log and Academic Integrity (10 pts) — `docs/ai-use-log.md`, plus the inline block at the top of every W2 artifact.
- Recorded Walkthrough of Problem Framing and Project Development (50 pts) — `walkthrough-script.md` and the recorded video file.

Total: 100 pts. Target: Level 1 (Advanced/Exceptional, 90–100%) on every dimension.

Signed:

Role

Student

Supervisor

End of approval brief.

Part 5 — Recorded walkthrough script (Appendix D)

This part of the consolidated submission is also available as a separate file in docs/02-proposal-package/ for graders who prefer per-document review. > AI-use disclosure. Drafted with Claude Sonnet 4.6 (Cowork desktop app). AI-drafted, student-revised. Key human-authored decisions in this document: every minute-budget allocation (the timing reflects the student’s own pacing rehearsal, not the AI’s first guess); the choice to make Decision Authorship a named segment rather than burying it in a closing note; the explicit ownership-of-trade-offs language in segments 5 and 7. Full audit trail: docs/ai-use-log.md.

Purpose

This is the script for the **recorded walkthrough** required by the W2 rubric (RU-02 dimension 6: “Recorded Walkthrough of Problem Framing and Project Development” — 50 of 100 points). The rubric asks for a clear, well-organized walkthrough of the problem framing, charter, scope, feasibility logic, and planned development path; it asks the student to explain key decisions, trade-offs, and next steps with confidence and strong technical ownership.

Production notes

- **Target length:** 14:30–15:30. Hard ceiling 16:00 — beyond that the recording risks signaling weak editing rather than strong substance.
- **Format:** Screen recording (1920×1080) with picture-in-picture webcam (top right, ~20% of frame). Camera is optional but recommended — the rubric language about “technical ownership” rewards a visible human.
- **Tools:** OBS Studio for capture; QuickTime or ScreenStudio acceptable. Microphone: any headset mic; record in a quiet room.
- **Filename:** safeexec-w2-walkthrough.mp4 (≤500 MB target; H.264, 8 Mbps, AAC audio).
- **Upload destination:** Canvas (media recording field) or YouTube unlisted with link in Canvas text entry. Confirm with course rules before recording.
- **Audio check:** Read the first 30 seconds aloud and play it back before recording the full take. The most common reason this kind of recording fails the rubric is unintelligible audio in segment 1.
- **One take vs. multi-take:** One take is acceptable; jump cuts at section boundaries are fine. Do not over-edit — the rubric rewards natural ownership over polish.

Materials on screen during the walkthrough

The “show on screen” cues below reference these source files. Have them open as tabs in the browser or panes in the IDE before recording starts.

1. README.md — opening tab
2. docs/02-proposal-package/proposal.md — primary reference
3. docs/02-proposal-package/project-plan.md — Gantt and dependency view

4. docs/02-proposal-package/wbs.md — work-breakdown table
5. docs/01-launch-packet/architecture-context.md — system-context diagram (Mermaid)
6. docs/01-launch-packet/screenshots/05-architecture-rendered.png — rendered PNG fallback
7. docs/ai-use-log.md — audit table
8. engineering-log.md — process evidence

A pre-rendered PNG of the Gantt chart should be saved at docs/02-proposal-package/screenshots/gantt.png before recording (export from mermaid.live using the source in project-plan.md §2).

Script

Total budget: **~15:00**. Each segment lists its time slot, on-screen cue, and narration. Read aloud at ~150 words per minute; the narration counts below assume that pace.

Segment 1 — Opening and roadmap (0:00 – 1:00, 1 min)

Show on screen: README.md open at the top, title visible.

Narration (~140 words):

“Hi, I’m Zixuan Liang. This is the Week 2 walkthrough for my CISC 699 applied project, SafeExec — a hardened, threat-modeled Python execution sandbox for LLM-agent tool-use. I’ll spend the next fifteen minutes walking through five things: first, the problem I’m solving and why it’s a real CS problem worth fourteen weeks; second, the proposal’s scope and the trade-offs I made to get there; third, the feasibility evidence across the MACP layers — machine, architectures, computational method, and API; fourth, the project plan and the critical-path chain I’m committing to; and fifth, my risk and ethics posture plus my AI-use discipline. I’ll close with what I’m asking my supervisor to approve and what comes next. The proposal document itself is in this repo at docs/02-proposal-package/proposal.md; everything I’ll show on screen is linked from there.”

On-screen action: Click into docs/02-proposal-package/proposal.md and scroll briefly to show the table of contents area.

Segment 2 — Problem framing and significance (1:00 – 3:00, 2 min)

Show on screen: `proposal.md` §§2–3 (Background and Problem statement). Optionally open OWASP and NIST source pages briefly in adjacent tabs.

Narration (~290 words):

“Here’s the problem. LLM-powered software agents — Claude Code, Code Interpreter, Cursor, Devin, an expanding open-source ecosystem — increasingly execute model-generated Python code as part of their tool-use loop. The executor is the layer where text becomes program execution on real infrastructure. In production, that layer is the one most often built quickly and revisited never. Most open-source agents today execute model output inside their own process tree, or inside a Docker container with default settings and a permissive network policy. Neither is a defensible posture when the code being executed was produced by a language model under conditions of possible prompt injection.”

“This is not a hypothetical concern. NIST AI 100-2’s 2025 update names tool-execution isolation as an under-specified mitigation surface. OWASP’s 2025 LLM Top 10 calls out sandboxed execution explicitly in LLM05, ‘Improper Output Handling,’ and in LLM06, ‘Excessive Agency.’ Real container-escape vulnerabilities continue to be discovered in widely deployed runtimes — `runc` CVE-2019-5736, `runc` CVE-2024-21626 — confirming that ‘default Docker’ isn’t a defensible posture.”

“What already exists: closed-source services like Anthropic’s analysis tool and OpenAI’s Code Interpreter address these risks behind opaque infrastructure. The open-source landscape has one strong commercial-backed option in E2B, which uses Firecracker microVMs, and a handful of lighter projects that ship convenience but not measured isolation strength. None of those open-source projects publishes a documented threat model paired with a reproducible adversarial benchmark. That gap — a measurement framework, not a new isolation primitive — is what SafeExec contributes.”

On-screen action: Highlight or scroll past the problem-statement paragraph in `proposal.md` §3.

Segment 3 — Candidates considered and selection rationale (3:00 – 5:00, 2 min)

Show on screen: `docs/01-launch-packet/candidate-projects.md` — the comparison matrix.

Narration (~290 words):

“I considered three candidate projects before committing to this one. Candidate A was a domain-specialized coding agent paired with a self-built benchmark — well-understood evaluation methodology, low technical risk, but one step removed from the kind of infrastructure work I want my portfolio to show.

Candidate B was a multi-model agent router optimizing cost, quality, and latency — interesting but a crowded product space and harder to differentiate as a research artifact. Candidate C is SafeExec — a hardened Python execution sandbox with two isolation back-ends and a curated adversarial benchmark. The comparison matrix is in docs/01-launch-packet/candidate-projects.md.”

“I chose C for four reasons, in roughly this order of weight. First, portfolio fit: I’m targeting software-engineering roles at frontier AI labs on agent-tooling teams, and SafeExec is a credible miniature of the kind of infrastructure those teams build. Second, rubric fit: the CISC 699 evaluation rubric weights ‘evaluation and evidence’ heavily, and C’s central contribution is an evaluation methodology — a forty-plus-program adversarial benchmark plus an A/B comparison across back-ends. That scores higher on the rubric levels that reward rigorous evaluation than A or B do, where the evaluation is borrowed from existing benchmarks. Third, ethics and broader impact: prompt injection, dual-use, agent misuse — those concerns are central to C, not tacked on. And fourth, the Python-only / no-network / no-GPU scope keeps both the threat model and the 14-week timeline honest. No model training. No specialized hardware. No fourteen-week scope creep.”

On-screen action: Briefly scroll the comparison matrix; pause on the “Worst-case failure mode” row.

Segment 4 — Scope and the scope-fence (5:00 – 6:30, 1:30 min)

Show on screen: proposal.md §8 (Scope) — show both the In-scope list and the Out-of-scope list.

Narration (~220 words):

“Scope. In-scope: Python 3.11 user code, synchronous request-response API, ephemeral per-request filesystem, the standard resource limits — CPU, memory, wall-clock, PIDs, file descriptors — two back-ends, hardened Docker and gVisor, on a Linux x86-64 host. Out-of-scope, explicitly: no multi-language support, no network access from inside the sandbox, no GPU, no persistence across requests, no multi-tenant security, no production-grade auth or TLS, no Windows or macOS, no Firecracker — Firecracker is a stretch only — no side-channel or hardware-level attacks, no nation-state adversaries. That out-of-scope list is the part most worth reading carefully, because the scope-fence policy in §8.3 says the in/out split is fixed from W3 through W8. The only sanctioned re-scoping moment is the Week 7 midpoint review. Outside that window, the default is no change. That’s deliberate — the worst-case failure mode for a project like this is scope creep into multi-language or persistence features, and the scope-fence is the explicit mitigation for that risk. I named the risk as R1 in the register, and the W7 checkpoint is the matching contingency.”

On-screen action: Highlight §8.3 (scope-fence policy) and pause briefly.

Segment 5 — Feasibility evidence across the MACP layers (6:30 – 9:00, 2:30 min)

Show on screen: `proposal.md` §9 (MACP framing) — scroll through 9.1, 9.2, 9.3, 9.4 in order. Have `engineering-log.md` open in an adjacent tab to show the W1 droplet inventory.

Narration (~370 words):

“Feasibility across the MACP layers — machine, architecture, computational method, API.”

“Machine. I provisioned a DigitalOcean Premium Intel droplet on May 17, two vCPU, four gigs of RAM, Ubuntu 22.04 LTS, kernel 5.15. Docker 29.5.0 and gVisor’s `runsc` release-20260511 are already installed. Total compute budget envelope is \$150 to \$250 across the term, well inside the charter’s \$400 ceiling. The droplet output is in the engineering log under the May 17 entry — `uname -r, cat /etc/os-release, docker version, runsc --version.`”

“Architecture. The system-context diagram is in `docs/01-launch-packet/architecture-context.md`. It commits the project to four load-bearing decisions: two back-ends not one, a single execution API, the threat model as a first-class deliverable, and external LLM only for the demo. The detailed component design — the Sentry-Gofer flows inside gVisor, the seccomp filter inside the Docker container, the cgroup hierarchy — is W4 work, captured in `docs/design/architecture.md`.”

“Computational method. The isolation primitives are well-established. Hardened Docker is OS-level container isolation with seccomp filters, AppArmor profiles, dropped capabilities, and cgroup-enforced limits. gVisor is user-space syscall interposition — its Sentry component implements its own kernel surface in Go, so the host kernel attack surface visible to the application is dramatically reduced. The evaluation method for the functional suite is deterministic output comparison — byte-equality of stdout and stderr against a reference CPython 3.11 interpreter, which is the same methodological model SWE-bench uses for its graders.”

“API integrations. Anthropic’s Claude API for the reference-agent demo only — a ≤50-line agent, budget ~\$30 to \$50. Docker Hub for base images. GitHub for source. HumanEval and MBPP, both permissively licensed, for the functional-corpus seed. gVisor’s apt repo for `runsc`. No other third-party SaaS in scope. Every one of these integrations has a named failure mode in §9.4 with a contingency.”

On-screen action: Briefly scroll the architecture diagram (PNG) and the MACP §9 table.

Segment 6 — Project plan, gates, and the critical path (9:00 – 11:30, 2:30 min)

Show on screen: `project-plan.md` §§2 (Gantt) and §4 (Critical path). Have `wbs.md` open in an adjacent tab.

Narration (~360 words):

“Project plan. Fourteen weeks, six phases, six explicit completion gates. The Gantt in `project-plan.md` shows them: G1 at W2 — this proposal, target approval by May 26. G2 at W4 — the design review for architecture, threat model, and evaluation plan. G3 at W5 — first end-to-end Python execution under the Docker baseline. G4 at W7 — midpoint demo and the only sanctioned scope-renegotiation point in the whole term. G5 at W11 — full report draft to my supervisor. G6 at W14 — final submission.”

“The critical path is G1 → G2 → G3 → G4 → W8 gVisor → G5 → G6. Any slippage on any link is a slip risk for the deliverable. I’ve built in two named relief valves. First, if hello-world execution misses the W5 gate, the contingency at June 19 activates — W6 becomes combined execution-plus-hardening, and that gets flagged as a W7 topic. Second, if the W7 midpoint shows fewer than fifty functional programs passing or fewer than ten adversarial programs working, the scope falls back to Docker-only and W8 redirects to adversarial-suite expansion. Those are pre-committed responses, named in §10.5 so the response isn’t improvised under pressure.”

“The WBS in `wbs.md` decomposes this into about thirty leaf tasks, each mapped to a week, to a deliverable, and to a success criterion in §7. The total estimate is about 272 student-hours across 14 weeks — about 17 to 20 hours a week. The W1 feasibility memo had it at 235; the 272 figure includes the contingency buffer. Every task maps to at least one success criterion. Every success criterion is covered by at least two tasks. The instructor’s self-check — ‘can every task in your plan be traced to a success criterion’ — is satisfied.”

On-screen action: Open the Gantt PNG (or scroll the Mermaid source); highlight the W5 and W7 gates; scroll briefly through the WBS table to show the SC column.

Segment 7 — Risks, ethics, and AI-use discipline (11:30 – 13:30, 2 min)

Show on screen: `proposal.md` §12 (Risk register), §14 (Ethics), §17 (AI-use); then briefly `docs/ai-use-log.md`.

Narration (~290 words):

“Risks. Twelve named in the register, each with likelihood, impact, mitigation, owner, and contingency. The four with the highest expected cost are: scope creep — addressed by the scope-fence in §8.3 and the W7 checkpoint; adversarial-suite quality — addressed by category lock at W3 and midpoint peer review; gVisor performance variability — addressed by the pre-registered benchmark protocol

with percentile reporting; and AI-use disclosure gaps — which leads me to the academic-integrity point.”

“Ethics. The adversarial suite is dual-use — it contains programs that probe for isolation failures. I’m publishing it anyway, because a benchmark that can’t be reproduced can’t be trusted, and the alternative is a claim no one can verify. I’m mitigating the dual-use concern in three ways: every adversarial program targets *only* SafeExec; the category-level taxonomy is published in the threat model, but per-program details are released only with the W14 artifact; and the report’s broader-impact section will explicitly state the trade-off.”

“AI-use. My Week 1 packet was graded 90 out of 100. The four-of-five dimensions at Level 1 was good; the AI-use log was at Level 2 — 15 of 20. The feedback was that the log was descriptive rather than audit-oriented. I restructured it on May 18 into the per-artifact audit-table format you can see in `docs/ai-use-log.md` — tool, purpose, prompt summary, authorship label, extent of student modification — plus an explicit enumeration of student-only decisions. Every artifact in this W2 package carries an inline disclosure block at the top, and the audit table now has entries for every new W2 document.”

On-screen action: Briefly open `docs/ai-use-log.md` to show the audit table; close it.

Segment 8 — What I’m asking, and what’s next (13:30 – 15:00, 1:30 min)

Show on screen: `approval-brief.md` open at the top; then close to `README.md`.

Narration (~220 words):

“What I’m asking my supervisor to approve at this hard stop. The one-page approval brief in `approval-brief.md` says: approved means scope, contribution framing, success criteria, project plan with gates, risk register, AI-use discipline, and reproducibility commitment. Conditional means the four supervisor questions from the W1 briefing: framing acceptability, open-source release timing, the reference-agent demo, and meeting cadence. Evidence expected by the W4 design review: requirements document, detailed architecture, threat model, evaluation plan, bibliography expanded to at least ten sources, and the engineering log kept current.”

“If the supervisor signals any of the four conditional items should change, the affected sections of `proposal.md` get revised before W3 begins, the revisions land in the engineering log, and we re-circulate.”

“Next steps if approval lands: I begin W3 lit-synthesis and requirements drafting on the assumption of approval. The next graded hard stop is the W7 midpoint, which is also the only sanctioned scope-renegotiation moment. Between now and then, the engineering log is the running record.”

“That’s the walkthrough. Thanks for watching. Everything I covered is in the repo at github.com/liangzixuan/cisc-699 under `docs/02-proposal-package/`.”

On-screen action: Close all tabs, leave README .md visible with the W2 section linked. Stop recording.

Pre-recording checklist

Run through this list within 24 hours before recording to avoid the most common failure modes:

- ☐ Microphone level tested; play back the first 30 seconds.
- ☐ All eight source files open as tabs in a fresh browser window or IDE.
- ☐ Notifications muted; calendar reminders silenced; Slack/Discord set to Do Not Disturb.
- ☐ Pre-rendered Gantt PNG saved at docs/02-proposal-package/screenshots/gantt.png.
- ☐ Architecture PNG visible (already at docs/01-launch-packet/screenshots/05-architecture-rendered.png).
- ☐ Camera framing rehearsed; lighting OK; background neutral.
- ☐ Water within arm's reach.
- ☐ One short rehearsal pass (read segments 1–3 aloud, time check).

Post-recording checklist

- ☐ Video runtime $\leq 16:00$; ideally 14:30–15:30.
 - ☐ Audio audible end-to-end; no dropouts.
 - ☐ Renamed safeexec-w2-walkthrough.mp4; saved under docs/02-proposal-package/recording/ (or to the cloud destination if file size is large).
 - ☐ Uploaded to Canvas (media field or YouTube unlisted link in text entry); upload confirmed.
 - ☐ AI-use log updated with a row for this recording (Authorship label: Student-only for the spoken delivery; AI-drafted, student-revised for this script).
 - ☐ Engineering log entry under W2 noting the recording date and any retakes.
-

End of walkthrough script.